Don't get locked up into avoiding lock-in

A significant share of architectural energy is spent on reducing or avoiding lock-in. That's a rather noble objective: architecture is meant to give us options and lock-in does the opposite. However, lock-in isn't a simple true-or-false matter: avoiding being locked into one aspect often locks you into another. Also, popular notions, such as open source automagically eliminating lock-in, turn out to be not entirely true. Time to have a closer look at lock-in, so you don't get locked up into avoiding it!

09 September 2019

Gregor Hohpe

Gregor is an architect who likes to tinker with IT systems and organizations. Formerly a technical director with Google Cloud, he's recently been appointed as Singapore Smart Nation fellow.

He enjoys making complex topics approachable without dumbing them down and never hesitates to spice things up with a pointed metaphor or anecdote. All views are his own. Follow him at @ghohpe or read more at ArchitectElevator.com

.

CONTENTS

ENTERPRISE ARCHITECTURE

One of an architect's major objectives is to create options. Those options make systems change-tolerant, so we can defer decisions until more information becomes available or react to unforeseen events. *Lock-in* does the opposite: it makes switching from one solution to another difficult. Many architects may therefore consider it their archenemy while they view themselves as the guardians of the free world of IT systems where components are replaced and interconnected at will.

> *Lock-in - an architect's archenemy?*

But architecture is rarely that simple - it's a business of trade-offs. Experienced architects know that there's more behind lock-in than proclaiming that it must be avoided. Lock-in has many facets and can even be the favored solution. So, let's get in the Architect Elevator to have a closer look at lock-in.

Open-source-hybrid-multi-cloud == lock-in free?

The platforms we are deploying software on these days are becoming ever more powerful - modern cloud platforms not only tell us whether our photo shows a puppy or a muffin, they also compile our code, deploy it, configure the necessary infrastructure, and store our data.

This great convenience and productivity booster also brings a whole new form of lock-in. Hybrid/multi-cloud setups, which seem to attract many architects' attention these days, are a good example of the kind of things you'll have to think of when dealing with lock-in. Let's say you have an application that you'd like to deploy to the cloud. Easy

enough to do, but from an architect's point of view, there are many choices and even more trade-offs, especially related to lock-in.

You might want to deploy your application in containers. That sounds good, but should you use AWS' Elastic Container Service (ECS) to run them? After all, it's proprietary to Amazon's cloud. Prefer Kubernetes? It's open source and runs on most environments, including on premises. Problem solved? Not quite - now you are tied to Kubernetes - think of all those precious YAML files! So you traded one lock-in for another, didn't you? And if you use a managed Kubernetes services such as Google's GKE or Amazon's EKS, you may also be tied to a specific version of Kubernetes and proprietary extensions.

If you need your software to run on premises, you could also opt for AWS Outposts, so you do have some options. But that again is proprietary. It integrates with VMWare, which you are likely already locked into, so does it really make a difference? Google's equivalent, freshly minted Anthos, is built from open-source components, but nevertheless a proprietary offering: you can move applications to different clouds - as long as you keep using Anthos. Now that's the very definition of lock-in, isn't it?

Alternatively, if you neatly separate your deployment automation from your application run-time, doesn't that make it fairly easy to switch infrastructure, reducing the effect of all that lock-in? Hey, there are even cross-platform infrastructure-as-code tools. Aren't those supposed to make these concerns go away altogether?

For your storage needs, how about AWS S3? Other cloud providers offer S3-compatible APIs, so can S3 be considered multi-cloud compatible and lock-in free, even though it's proprietary? You could also wrap all your data access behind an abstraction layer and thus localize any dependency. Is that a good idea?

It looks like avoiding lock-in isn't quite so easy and might even get you locked up into trying to escape from it. To highlight that cloud architecture is fun nevertheless, I defer to Simon Wardley's take on hybrid cloud.

Shades of lock-in

> *Lock-in isn't an all-or-nothing affair.*

Elevator Architects (those who ride the Architect Elevator up and down) see shades of gray where many only see black and white. When thinking about system design, they realize that common attributes like lock-in or coupling aren't binary. Two systems aren't just coupled or decoupled just like you aren't simply locked into a product or not. Both properties have many nuances. For example, lock-in breaks down into numerous dimensions:

- **Vendor Lock-in**: This is the kind that IT folks generally mean when they mention "lock-in". It describes the difficulty of switching from one vendor to a competitor. For example, if migrating from Siebel CRM to SalesForce CRM or from an IBM DB2 database to an Oracle one will cost you an arm and a leg, you are "locked in". This type of lock-in is common as vendors generally (more or less visibly) benefit from it. This lock-in includes commercial arrangements, such as long-term licensing and support agreements that earned you a discount off the license fees back then.
- **Product Lock-in**: Related, but different is being locked into a product. When migrating from one vendor's product to another vendor's, you are usually changing both vendor and product, so the two are easily conflated. Open source products may avoid the vendor lock-in, but they don't remove product lock-in: if you are using Kubernetes or Cassandra, you are certainly locked into a specific product's APIs, configurations, and features. If you work in a professional (and especially enterprise) environment, you will also need commercial support, which will again lock you into a vendor contract - see above. Heavy customization, integration points, and proprietary extensions are forms of product lock-in: they make it difficult to switch to another product, even if it's open source.
- **Version lock-in**: Besides being locked into a product, you may even be locked into a specific version. Version upgrades can be costly if they break existing customizations and extensions you have built (SAP, anyone?). Other version upgrades essentially require you to rewrite your application - AngularJS vs. Angular 2 comes to mind. To make matters worse, version lock-in propagates: a certain product version may require a certain (often outdated) operating system version and so on, which turns any migration attempt into a Yak-shaving exercise. You feel this lock-in particularly badly when a vendor decides to deprecate your version or discontinues the whole product line: you have to choose between being out of support or doing a major overhaul. And things can get even worse, for example, if a

major security vulnerability is found in your old version and patches aren't provided.

- **Architecture lock-in**: You may also be locked into a specific kind of architecture. For example. when you use Kubernetes extensively, you are likely building small-ish services that expose APIs and can be deployed as containers. If you want to migrate to a serverless architecture, you'll want to change the granularity of your services closer to single functions, externalize state management, utilize an event-architecture, and probably a few more things. Such changes aren't minor, but imply a major overhaul of your application architecture.
- **Platform lock-in**: A special flavor of product lock-in is being locked into a platform, especially cloud platforms. Such platforms not only run your applications, but they may also hold your user accounts and associated access rights, security policies, infrastructure segmentations and many more aspects. They also provide application-level services such as storage or machine learning services, which are generally proprietary. Staying away from these services might seem like a way to reduce platform lock-in but it'd negate one of the major motivations for moving to the cloud in the first place. Non-software people call this finding yourself between a rock and a hard place.
- **Skills lock-in**: As your developers are becoming familiar with a certain type of product or architecture, you'll have skills lock-in: it'll take you time to re-train (or hire) developers for a different product or technology. As skills availability is one of the major constraints in today's IT shops, this type of lock-in is very real. Some niche enterprise products have a particularly limited supply of developers, causing your cost for developers to go up. This effect is particularly visible for products that employ custom languages or, somewhat ironically, for “config only” / no-code frameworks.
- **Legal lock-in**: You may be locked into a specific solution for legal reasons, such as compliance. For example, you might not be able to migrate your data to another cloud provider's data center if it's located outside your country. Your software provider's license may also not allow you to move your systems to the cloud even though they'd run perfectly fine. If you decide to do it anyway, you'll be in violation of licensing terms. Legal aspects permeate more facets of engineering than we'd commonly assume: your small-engine air craft is likely to be powered by an engine that was designed back in the 1970s and burns heavily leaded fuel: new engine designs face high legal liabilities.
- **Mental Lock-in**: The most subtle, but also the most dangerous type of lock-in is the one that affects your thinking. After working with a certain set of vendors and architectures, you are likely to absorb assumptions into your decision making,

which may lead you to reject alternative options. For example, you may reject scale-out architectures as inefficient because they don't scale linearly (you don't get twice the performance when doubling the hardware). While technically accurate, this way of thinking ignores the fact that scalability, not efficiency, is the main driver. Or you may resent short release cycles as you have observed frequent changes leading to more defects. And surely you've been told that coding is expensive, time-consuming, and error-prone, so you'd be better off doing everything via configuration.

> *Open source software isn't a magic cure for lock-in.*

In summary, lock-in is far from an all-or-nothing affair, so understanding the different flavors can help you make more conscious architecture decisions. The list also debunks common myths, such as using open source source software magically eliminating lock-in. Open source can reduce vendor lock-in, but most of the other types of lock-in remain. This doesn't mean open source is bad, but it isn't a magic cure for lock-in.

Making better decisions using models

Experienced architects not only see more shades of gray, they also practice good decision discipline. That's important because we are much worse decision makers than we commonly like to believe - a quick read of Kahneman's Thinking, Fast and Slow is in order if you have any doubt.

One of the most effective ways to improve your decision making is to use models. Even, or especially, simple models are surprisingly effective at improving decision making:

> *Simple but evocative models are the signature of the great scientist, but over-elaboration and over-parameterization is often the mark of mediocrity.*
>
> -- *George Box*

That's why you shouldn't laugh at the famed two-by-two matrix that's so beloved by management consultants. It's one of the simplest and therefore most effective models as we shall soon discover.

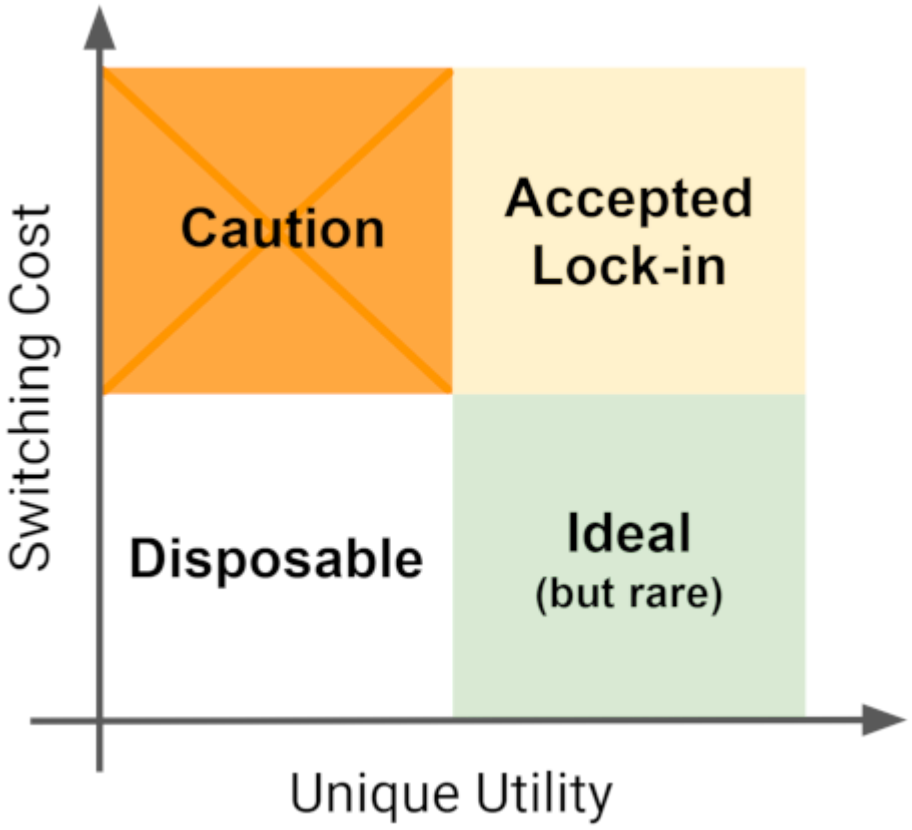
> *The more uncertain the environment, the more structured models can help you make better decisions.*

There's a second important point about models: a common belief tells us that in face of uncertainty you pretty much have to “shoot from the hip” - after all everything is in flux, anyway. The opposite is actually true: our generally poor decision making only gets worse when we have to deal with many interdependencies, high degrees of uncertainty, and small probabilities. Therefore, this is where models help the most to bring much needed structure and discipline into our decision-making. Deciding on whether and to what degree to accept lock-in falls well into this category, so let's use some models.

Lock-in as a two-by-two matrix

A simple model can help us get past the “lock-in = bad” stigma. First, we have to realize that it's difficult to not be locked into anything, so some amount of lock-in is inevitable. Second, we may happily accept some amount of lock-in if we get a commensurate pay-off, for example in form of a unique feature or utility that's not offered by competitive products.

Let's express these factors in a very simple model - a two-by-two matrix:



The matrix outlines our choices along the following axes:

- switching cost (aka “lock-in”): how difficult will be for us to move to another solution?
- unique utility: how much are we gaining from the solution compared to alternatives?

We can now consider each of the four quadrants:

- **Disposable**: Components that don't have a unique utility and are easy to replace are the ones we may have to worry about the least. We can leave them as is or, if we face any issues, we can easily replace them. Not a bad place to be for run-of-the-mill stuff. For example, most developer IDEs (EMACS likely being a notable exception!) fall into this category: mix and match as you please and don't get too attached to them. Cloud storage for all your photos and other personal data has also largely moved your smartphone device into this box, but more on this later.
- **Accepted Lock-in**: across the diagonal are the components that lock you into a specific product or vendor, but in return give you a unique feature or utility. While we generally prefer less lock-in, this trade-off may well be acceptable. You may use a product like Google Cloud BigQuery or AWS Bare Metal Instances, knowing well that you are locked in, having made a conscious decision based on the pay-off

you're getting. For a small application, you may also happily use native AWS services because a migration is unlikely and the reduction in development and operations effort is very welcome.

- **Caution**: the least favorable box is the one that locks you in but doesn't give you a lot of unique utility. Your traditional relational database may fall into this box - does using any proprietary database really increase your revenue? Not really. However, migrating off can be a lot of effort, so you better be sure that there's a low likelihood you're going to need to do that. If you selected a particular hardware for your embedded system that you launched into outer space, that's likely OK - the chances of a migration are rather low.
- **Ideal**: the best stuff is the one that gives you a unique utility but at the same time is easy to switch away from. While that sounds like the ideal to strive for, you'll have to acknowledge that the box is a bit of an oxymoron: if a solution gives you *unique* utility, per definition competitive products won't have it, making a migration difficult. S3 may be a suitable example for this category - multiple cloud vendors have adopted the same APIs, making a switch to let's say GCP relatively easy. Still, each implementation has some distinct advantages regarding locality, performance, etc. To protect this kind of portability across differentiated products it's important that we don't allow APIs to be copyrighted or patented.

While the model is admittedly simple, placing your software (and perhaps hardware) components into this matrix is a worthwhile exercise. It not only visualizes your exposure but also communicates your decisions well to a variety of stakeholders.

For an every-day example of the four quadrants, you may have decided to use following items, which give you varying amounts of lock-in and utility (counter-clockwise from top-right):

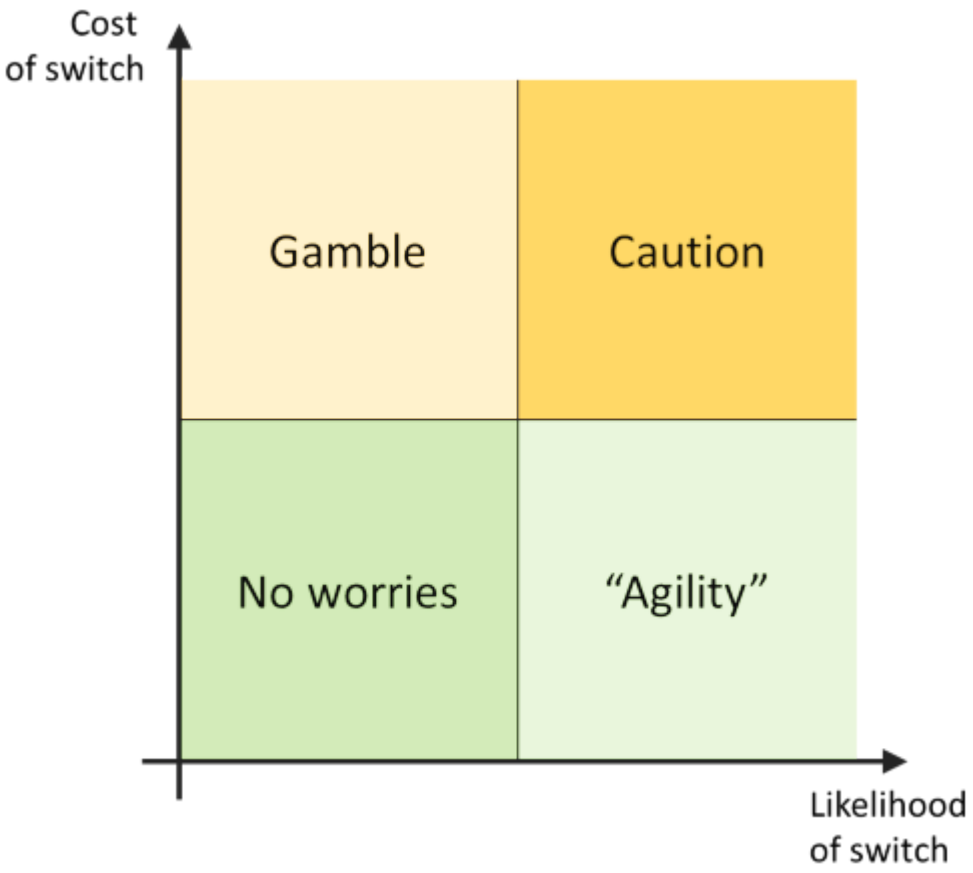
- Your beloved **iPhone** locks you into a vendor ecosystem, but it also gives unique utility, so you are likely OK to have this **Accepted Lock-in**.
- Your **mobile provider contract** locks you into a single network, but doesn't really provide much utility over other networks. It's better to exercise **Caution**.
- Your **phone charger** has a standard connector. Sadly, many iPhones don't, but luckily an adapter cable places still makes this gadget **Disposable**.
- Many of your **apps, such as messaging,** give you utility, such as having your friends on it, but they are still designed to make it easy to switch, for example by using your phone's contact list. That's **Ideal**.

> *A unique product feature doesn't always translate into unique utility for you.*

One word of caution on the *unique utility*: every vendor is going to give you some form of unique feature - that's how they differentiate. However, what counts here is whether that feature translates into a concrete and unique *value* for you and your organization. For example, some cloud providers run Billion-user services over their amazing global network. That's impressive and unique, but unlikely to be a utility for the average enterprise who's quite happy to serve 1 million customers and may be restricted to doing business in a single country. Some people still buy Ferraris in small countries with strict speed limits, so apparently not all decision making is entirely rational, but perhaps a Ferrari gives you utility in more ways than a cloud platform can.

The actual cost of lock-in

Because this simple matrix was so useful, let's do another one. The previous matrix treats *switching cost* as a single element (or dimension). A good architect can see that it breaks down into two dimensions:



The matrix differentiates between the cost of making the switch from the likelihood that you'll have (or want) to make the switch. Things that have a low likelihood and a low cost shouldn't bother you much while the opposite end, the ones with high switching cost and a high chance of switch, are no good and should be addressed. On the other diagonal, you are taking your chances on those options that will cost you, but are unlikely to occur - that's where you'll want to buy some insurance, for example by limiting the scope of change or by padding your maintenance budget. You could also accept the risk - how often would you really need to migrate off Oracle onto DB2, or vice versa? Lastly, if switches are likely but cheap, you achieved agility - you embrace change and designed your system for low cost of executing it. Oddly, this quadrant often gets less attention than the top left despite many small changes adding up quickly. That's our poor decision making at work: the unlikely drama gets more attention because *what if*!

When discussing the likelihood of lock-in, you'll want to consider a variety of scenarios that'll make you switch: a vendor may go out of business, raise prices, or may no longer be able to support your scale or functional needs. Interestingly, the desire to reduce lock-in sometimes comes in form of a negotiation tool: when negotiating license renewals you can hint your vendor that you architected your system such that switching away from their product is realistic and inexpensive. This may help you

negotiate a lower price because you've communicated that your BATNA - your Best Alternative To a Negotiated Agreement is low. This is an architecture option that's not really meant to be used - it's a deterrent, sort of like a stockpile of weapons in a cold war. You might be able to fake it and not actually reduce lock-in, but you better be a good poker player in case the vendor calls your bluff, e.g. by chatting with your developers at the water cooler.

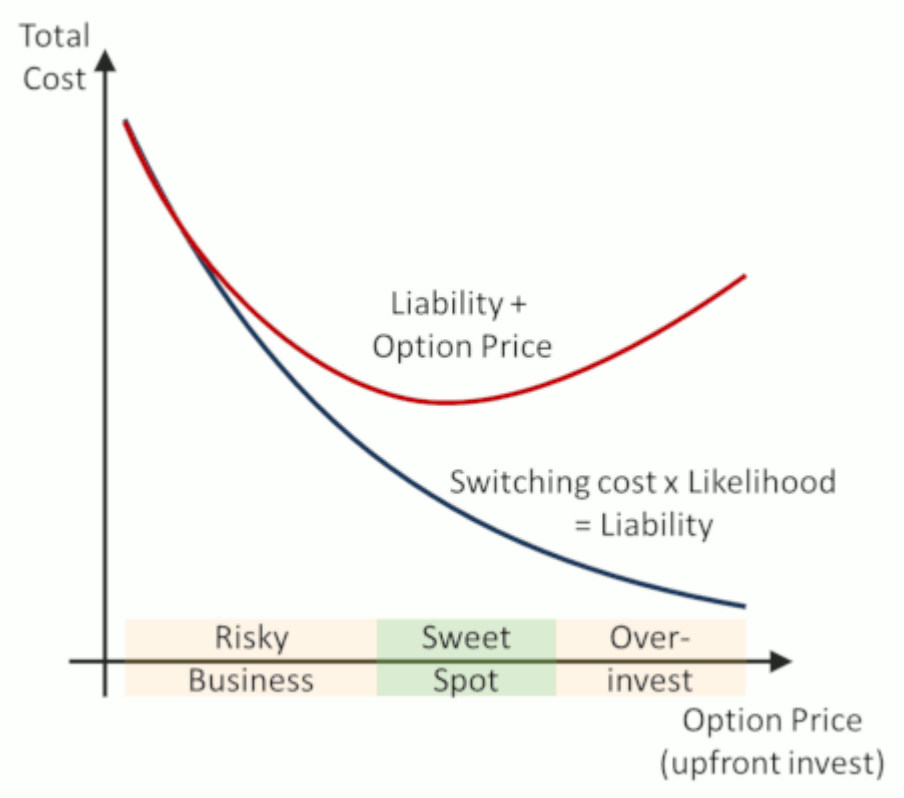
Reducing lock-in: The strike price

Pulling in our options analogy from the very beginning once more, if avoiding lock-in gives you options, then the cost of making the switch is the option's strike price: it's how much you pay to execute the option. The lower the switching cost you want to achieve, the higher is the option's value and therefore the price. While we'd dream of having all systems in the “green boxes” with minimal switching cost, the necessary invest may not actually pay off.

> *Minimizing switching costs may not be the most economical choice.*

For example, many architects favor not being locked into a database vendor or cloud provider. However, how likely is a switch really? Maybe 5%, or even lower? How much will it cost you to bring that switching cost down from let's say $50,000 (for a semi-manual migration) to near zero? Likely a lot more than the $2,500 ($50,000 x 5%) you can expect to save. Therefore, minimizing the switching cost isn't the sole goal and can easily lead to over-invest. It's the equivalent of being over-insured: paying a huge premium to bring the deductible down to zero may give you peace of mind, but it's often not the most economical, and therefore, rational, choice.

A final model (for once not a matrix) can help you decide how much you should invest into reducing the cost of making a switch. The following diagram shows your liability, defined as the product of switching cost times the likelihood that it occurs in relation to the up-front invest you need to make (blue line).



By investing in options, you can surely reduce your liability, either by reducing the likelihood of a switch or by reducing the cost of executing it. For example, using an Object-relational Mapping (ORM) framework like Hibernate is a small investment that can reduce database vendor lock-in. You could also create a meta-language that is translated into each database vendor's native stored procedure syntax. It'll allow you to fully exploit the database's performance without being dependent, but it's going to take a lot of up-front effort for a relatively unlikely scenario.

The interesting function therefore is the red line, the one that adds the up-front invest to the potential liability. That's your total cost and the thing you should be minimizing. In most cases, with increasing up-front invest, you'll move towards an optimum range. Additional investment into reducing lock-in actually leads to higher total cost. The reason is simple: the returns on investment diminish, especially for switches that carry a small probability. If we make our architecture ever-so-flexible, we are likely stuck in this zone of over-investment. The Yagni (you ain't gonna need it) folks may aim for the other end of the spectrum - as so often, the trick is to find the happy medium.

The total cost of avoiding lock-in

Now that we have a pretty good grip on the costs and potential pay-offs of being locked in, we need to have a closer look at the total cost of *avoiding* lock-in. In the previous model we assumed that avoiding lock-in is a simple cost. In reality, though, this cost can be broken down into several components:

> *Complexity can be the biggest price you pay for reducing lock-in.*

- **Effort**: This is the additional work to be done in terms of person-hours. If we opt to deploy in containers on top of Kubernetes in order to reduce cloud provider lock-in, this item would include the effort to learn a new tool, write Docker files, configure Kubernetes, etc.
- **Expense**: This is the additional cash expense, e.g. for product licenses, to hire external providers, or to attend KubeCon.
- **Underutilization**: This indirect cost occurs because avoiding lock-in often disallows you from using vendor-specific features. As a result, you get less utility out of the software you use. This in turn can mean more effort for you to build the missing features or it can cause a weakness in your product.
- **Complexity**: Complexity is a core element of the equation, and too often ignored. Many efforts to reduce lock-in introduce an additional layer of abstraction: JDBC, Containers, common APIs. While all useful tools, such a layer adds another moving part, increasing the overall system complexity. This in turn increases the learning effort for new team members and the chance of systemic errors.
- **New Lock-ins**: Avoiding one lock-in often comes at the expense of another one. For example, you may opt to avoid AWS CloudFormation and instead use Hashicorp's Terraform or Pulumi, which both support multiple cloud providers. However, now you are locked into another product from an additional vendor and need to figure out whether that's OK for you.

When calculating the cost of avoiding lock-in, an architect should make a quick run down this list to avoid blind spots. Also, be aware that attempts at avoiding lock-in can be leaky, very much like leaky abstractions. For example, Terraform is a fine tool, but its scripts use many vendor-specific constructs. Implementation details thus “leak” through, rendering the switching cost from one cloud to another decidedly non-zero.

Bringing it back together

With so much theory, let's look at a few concrete examples.

Deploying Containers

I worked with a company who packages much of their code into Docker containers that they deploy to AWS ECS. Thus they are locked into AWS. Should they invest into replacing their container orchestration with Kubernetes, which is open source? Given that feature velocity is their main concern and the current ECS solution works well for them, I don't think a migration would pay off. The likelihood of having to switch to another cloud provider is low and they have “bigger fish to fry”.

Recommendation: accept lock-in.

Relational database access

Many applications use a relational database that can be provided by numerous vendors and open source alternatives. However, SQL dialects, stored procedures, and bespoke management consoles all contribute to database lock-in. How much should you invest into avoiding this lock-in? For most languages and run-times common mapping frameworks such as *Hibernate* provide some level of database neutrality at a low cost. If you want to further minimize your strike price, you'd also need to avoid SQL functions and stored procedures, which may make your product less performant or require you to spend more on hardware.

Recommendation: use low-effort mechanisms to reduce lock-in. Don't aim for zero switching cost.

Migrating to the cloud

Rather than switching from one database vendor to another, you may be more interested in moving your application, including its database, to the cloud. Besides technical considerations, you'll need to be careful with some vendors' licensing agreements that may make such a move uneconomical. In these cases, it's wise to opt for an open source database.

Recommendation: select an open source database if it can meet your operational and support needs, but accept some degree of lock-in.

Multi-cloud

Many enterprises are fascinated the idea of portable multi-cloud deployments and come up with ever more elaborate and complex (and expensive) plans that'll ostensibly keep them free of cloud provider lock-in. However, most of these approaches negate the very reason you'd want to go to the cloud: low friction and the ability to use hosted services like storage or databases.

Recommendation: Exercise caution. Read my article on multi-cloud.

Architecture at the speed of thought

It may seem that one can put an enormous amount of time contemplating lock-in. Some may even dismiss our approach as “academic”, a word which I repeatedly fail to see as something bad because that's where most of us got our education. Still, isn't the old black-or-white method of architecture simpler and, perhaps, more efficient?

> *Architectural thinking is actually surprisingly fast if you focus and stick to simple models.*

In reality thinking actually happens extremely fast. Running through all the models shown in this article may really just take a few minutes and yields well-documented decisions. No fancy tooling besides a piece of paper or a whiteboard is required. The key ingredient into fast architectural thinking is merely the ability to focus.

Compare that to the effort to prepare elaborate slide decks for lengthy steering committee meetings that are scheduled many weeks in advance and usually don't have anyone attend who has the actual expertise to make an informed decision.

Elevator Architects prefer to spend their time thinking over waiting for meetings.

Acknowledgments

I'd like to thank the following individuals for their valuable feedback and input into this article: Manlio Grillo, Michael Plöd, and Michele Danieli, and Scott Davis.

▶ Significant Revisions